Continuous Integration Strategy for Research Compendia: A Tiered Model for zzcollab

2026-05-05 17:54 PDT

Abstract

The default Continuous Integration (CI) configuration in zzcollab inherits conventions developed for distributing R packages to CRAN. Those conventions are tangentially related to the framework's actual purpose, which is the construction and verification of reproducible research compendia. This white paper identifies the category error embedded in the current `r-package.yml` workflow, characterizes the regressions a research compendium plausibly cares about, and proposes a three-tiered CI model that aligns the checking strategy with the framework's reproducibility claims. The recommendation is not to abandon `R CMD check`, but to demote it from sole gatekeeper to one of three independent jobs, each serving a distinct verification purpose with appropriate cadence and cost.

1. Problem Statement

A zzcollab project is a research compendium. Its success criterion is that an analyst on a fresh machine, possibly years after original publication, can clone the repository, build the documented Docker image, restore the locked package versions, and reproduce the figures, tables, and report contained in the work. The Five Pillars of zzcollab (Dockerfile, renv.lock, .Rprofile, source code, data) jointly serve this goal.

The current default CI workflow runs two operations:

1. `renv::restore()` against the project lockfile, inside a pinned container.
2. `R CMD check` against the R package metadata.

These operations are necessary but insufficient. They verify that the package metadata is well-formed and that the locked packages are installable. They do not verify that the analysis itself runs to completion or that the report renders correctly. They also conflate two distinct verification concerns into a single brittle step, with the result that infrastructure issues (network state, cache state, URL mismatches) can produce false negatives that do not reflect genuine project unhealth.

This paper argues that the conventional R-package CI idiom is the wrong default for a research-compendium framework. It proposes a model in which CI is structured around the regressions a compendium author actually cares about, rather than around the conventions of CRAN distribution.

2. The Category Error

`R CMD check` is a tool for verifying that an R package, taken as an artifact, is well-formed for distribution. It checks that documentation matches code, that exported names are documented, that examples run without error, that the NAMESPACE is consistent, and that tests pass. All of these are necessary properties for a CRAN-bound package and remain useful for any R codebase.

A research compendium has a different success criterion. The artifact of interest is not a tarball deposited to CRAN; it is a body of work whose inputs, methods, and outputs travel together and can be regenerated on demand. The questions a reader asks of a compendium are: does the analysis pipeline run end-to-end, does the report render to its expected form, are the locked dependencies still recoverable, are the declared package imports actually used, and are the actually-used packages declared.

`R CMD check` does not directly answer any of these. It is possible for `R CMD check` to pass cleanly while the analysis script errors at the first chunk and the rendered report is broken. The two checks are substantively different.

The current zzcollab default CI runs the check that is conventional for R packages, not the check that maps to the framework's purpose. This is the category error.

3. What CI Should Detect

A useful frame for CI design is to ask: what regression would I be unhappy to ship, and what test would catch it? Different regressions require different tests, and most do not subsume each other.

The table below enumerates the regressions a zzcollab compendium author plausibly cares about, the test that detects each, and a rough cost.

Regression	Detection mechanism	Cost
Code change broke a function or test	`R CMD check` plus the test runner	medium
New `library()` call without DESCRIPTION update	`renv::status()` or zzrenvcheck audit	cheap
Lockfile drifted from declared dependencies	`renv::status()`	cheap
Fresh clone cannot restore the environment	`docker build` plus `renv::restore()` from clean cache	medium
Report no longer renders end-to-end	`rmarkdown::render()` or `quarto render` in CI	high
Output figures or tables changed unexpectedly	hash or diff outputs against a baseline	high, fragile
Upstream dependency broke compatibility	restore against current CRAN, no date pin	cheap, schedulable
Snapshot date too old to download packages	timed restore on schedule	cheap

Conventional R-package CI addresses only the first row. The remaining rows describe regressions that matter to a research compendium and are either covered weakly or not at all by the current workflow.

4. The Five Pillars and Their CI Coverage

Mapping the zzcollab Five Pillars to CI checks reveals where the current workflow is strong, where it is weak, and where it is silent.

Dockerfile. The environment definition. CI should verify that the container image either builds from `Dockerfile` or pulls from the declared registry. A failed build is the loudest signal that the environment definition has decayed.

renv.lock. The package pin. CI should verify two distinct properties: that the lockfile restores cleanly against the URLs it records, and that the lockfile matches both DESCRIPTION and the actual `library()`/`pkg::` references in source. The second property is what zzrenvcheck exists to enforce. The current workflow attempts the first property and ignores the second.

.Rprofile. The session bootstrap. Difficult to test in isolation. Best verified indirectly through successful R startup and successful restore.

Source code. The conventional target of `R CMD check`. The current workflow handles this adequately when the prior steps succeed.

Data. Provenance, file hashes, and schemas. Largely a documentation discipline rather than a CI concern, although CI can hash inputs and outputs as a regression check.

The Pillar that receives the least CI coverage today is the one closest to the framework's purpose: end-to-end reproduction of the analysis. If `analysis/report/report.Rmd` or `index.qmd` renders successfully from a clean environment, the compendium is much closer to verified than if `R CMD check` alone passes.

5. Proposed Tiered CI Model

CI should be structured as three independent jobs, each gating a different class of regression at an appropriate cadence.

Tier 1: Environment Integrity

Cheapest job. Runs on every push to any branch. Verifies that the framework remains internally consistent.

```
renv::restore(prompt = FALSE)
status <- renv::status()
if (!isTRUE(status$synchronized)) {
  quit(status = 1)
}
zzrenvcheck::check_packages()
```

`renv::restore()` confirms the lockfile is restorable. `renv::status()` confirms the lockfile, DESCRIPTION, and source code agree on what packages are in use; the explicit `quit(status = 1)` is required because `renv::status()` reports findings without raising an error, and a tier that does not fail when inconsistency is detected does not catch the regression it is meant to catch (see Section 9.7).

`zzrenvcheck::check_packages()` adds a finer-grained audit (orphaned imports, undeclared usage). Expected duration under thirty seconds in steady state, with a warm cache. This is the check that directly addresses the framework's reproducibility claim and is currently the least enforced part of the workflow.

Tier 2: Code Correctness

Conventional R-package check. Runs on pull requests. Catches code-level regressions.

```
rcmdcheck::rcmdcheck(args = '--no-manual', error_on = 'error')
tinytest::test_package(pkg)
```

This is the current workflow's purpose. Useful, narrow, and well understood. Expected duration one to five minutes.

Tier 3: End-to-End Reproduction

The check that maps directly to the compendium's success criterion. Runs on `main` after merge or on a nightly schedule. Catches reproducibility regressions.

```
rmarkdown::render('analysis/report/report.Rmd')
# or
quarto::quarto_render('index.qmd')
```

Optionally, hash the produced outputs and fail if the hash differs from a recorded baseline. The cost depends on the analysis (potentially many minutes), so this tier is not appropriate for every push, but it is the single check that verifies the framework's promise.

6. Implications for Reliability

A workflow's pass rate is partly a function of what it is checking. A workflow that runs `echo hello` will pass routinely and tell the author nothing. A workflow that runs the full analysis end-to-end will fail more often, but its failures will be informative.

The goal is not "passes routinely" in isolation. The goal is "passes routinely when the project is healthy, and fails informatively when the project is broken." The current v2.4.0-A workflow fails for infrastructure reasons (curl absence, lockfile URL mismatch, GitHub Actions cache state) rather than for project-health reasons. That is the worst of both worlds: false-negative noise that conveys no information about the research.

Splitting the workflow into the three tiers proposed above has two benefits. First, each tier becomes reliable on its own terms; an infrastructure issue in Tier 3 does not prevent Tier 1 from telling the author that the lockfile drifted. Second, the failure mode of each tier is informative; a Tier 1 failure means the framework is internally inconsistent, a Tier 2 failure means code or tests broke, a Tier 3 failure means the analysis pipeline regressed.

7. Comparison to v2.2.0

The earlier v2.2.0 workflow template, retained in some zzcollab projects, already gestures toward this tiered structure. It defines a `check` job plus three conditional jobs (`render-report`, `validate-data`, `render-blog`) that fire when `analysis/report/report.Rmd`, `analysis/data/`, and `index.qmd` are present respectively. The conditional jobs implement an elementary version of Tier 3.

The v2.4.0-A simplification dropped these conditional jobs in favor of a single renv-restore-and-check pipeline. From a CI-strategy perspective this was a regression in scope, not merely an implementation refactor. The newer workflow is technically more sophisticated (renv-aware, pinned container, separate CI tools library) but covers strictly less of the framework's reproducibility surface.

Empirically, this tradeoff produced worse outcomes. Across nine projects audited in 2026-05, the v2.2.0 workflow passes in three of four projects; the v2.4.0-A workflow passes in one of five. The newer workflow's narrower scope did not buy the reliability the trade implied.

8. Recommendations

The recommendation is not to revert to v2.2.0, which has its own reliability issues, nor to abandon the renv-based approach, which is correct for the framework's purpose. The recommendation is to restructure the canonical zzcollab `r-package.yml` template along the following lines.

Provide three jobs, each of which can be enabled independently:

1. `validate`: a fast Tier 1 job that runs on every push. Restores the lockfile, runs `renv::status()`, and runs the zzrenvcheck audit if the project declares zzrenvcheck as a dev dependency. Should pass under thirty seconds with a warm cache.
2. `check`: the Tier 2 job. Runs on pull requests. Performs the conventional `R CMD check` plus test execution. The current workflow collapsed into this job alone.
3. `render`: the Tier 3 job. Conditionally enabled when `analysis/report/report.Rmd` or `index.qmd` is present. Runs on `main` after merge or on a weekly schedule. Renders the document and uploads the output as a build artifact.

Independent additional changes that improve reliability of any tier:

- Pin `R$Repositories.URL` in `renv.lock` to a date-stamped Posit Package Manager URL with the appropriate OS segment, rather than the default unqualified `cran/latest`. This eliminates source-build fallback, version drift between restores, and the RSPM-tag-without-URL resolution problem.
- Set `ZZCOLLAB_AUTO_RESTORE=false` in the workflow environment so the workflow controls restore explicitly rather than racing the `.Rprofile` auto-restore branch.
- Add `curl` and `ca-certificates` to the system-deps apt line as a defensive measure against the renv curl-CLI fallback warning, even though the warning has been verified to be non-fatal for download success.

These changes are independent. The lockfile URL fix alone substantially improves reliability of the current single-job workflow. The tiered restructure is the larger change and addresses the underlying category error.

9. Lessons from Implementation

The recommendations in Section 8 were tested by application to two projects: zzobj2fig (a tool package in the zzcollab framework) and mci (a research compendium). The implementation surfaced findings not anticipated by the original analysis. They are recorded here as diagnostic and design lessons rather than completion claims; at the time of writing, the new workflows have been authored but their CI runs have not yet been observed end-to-end.

9.1 The lockfile URL is the silent failure mode

The default lockfile written by `renv::init()` in a zzcollab project records a single repository entry:

```
{"Name": "CRAN", "URL": "https://packagemanager.posit.co/cran/latest"}
```

The URL `cran/latest`, without the `__linux__/<distro>/` segment, serves source tarballs only. Posit Package Manager only returns binaries when the URL contains the OS-specific path. Consequently, every `renv::restore()` call source-compiles every package, even inside a container whose OS is supported by Posit's binary repository.

Two further issues compound:

1. The base name `latest` is a moving target. On any restore, renv may decide that the lockfile-recorded version is unobtainable and "repair" the dependency by installing whatever CRAN currently serves. In one observed restore, twenty-two packages were silently upgraded relative to the lockfile and three new packages were added. The lockfile failed its core function on a single round trip.
2. The lockfile records a substantial fraction of packages with `Repository: "RSPM"` but does not include an `RSPM` entry in the `Repositories` block. Renv falls back to `getOption('repos')` to resolve those packages, with behavior that varies by renv version.

The fix is to pin `R$Repositories` at init time to a date-stamped Posit binary URL with the correct OS segment, and to add a parallel `RSPM` entry pointing to the same URL.

9.2 The .Rprofile auto-restore races the workflow

The zzcollab `.Rprofile` template fires `renv::restore(prompt = FALSE)` on R session start when `ZZCOLLAB_AUTO_RESTORE` is unset or `true`. The CI workflow also invokes `renv::restore()` explicitly. Both mechanisms can fire, sometimes in inconsistent orders relative to other workflow steps. Setting `ZZCOLLAB_AUTO_RESTORE=false` in the workflow environment hands restore control to the workflow and removes the race.

9.3 Misleading symptoms in CI logs

A prominent warning in failed CI logs read

```
curl does not appear to be installed; downloads will fail.
```

This warning anchored an early diagnosis that attributed the failure to a missing `curl` command-line binary in the rocker container. The diagnosis was incorrect. Renv warns about the absence of the curl CLI but transparently falls back to R's `utils::download.file()`, which uses libcurl (the C library, present via `libcurl4-openssl-dev`) and completes downloads successfully. The actual failure was a version mismatch: CI attempted to install `Rcpp 1.1.1-1` (a binary build identifier) while the lockfile pinned `Rcpp 1.0.14`, with the mismatch produced by interaction between the GitHub Actions cache, the in-lockfile source URL, and the `.Rprofile` repository override.

The lesson is procedural: when a CI failure log presents multiple candidate causes, do not anchor on the most visually prominent one. Trace the full call chain. A warning is not a failure unless its absence would have prevented the failure.

9.4 Tool packages and compendia have different CI needs

zzcollab generates project scaffolding for both tool packages (zzobj2fig is one example: a package whose purpose is to expose functions for use elsewhere) and research compendia (mci is one example: a package whose primary deliverable is a rendered analysis report). The framework's default `r-package.yml` workflow treats both project types identically.

The two project types have different CI needs. A tool package's CI should answer "does the package install, pass `R CMD check`, and run its tests?" The community's canonical answer is the `r-lib/actions/setup-renv@v2` pattern (Section 10.1) without a container. A compendium's CI should answer the same questions plus "does the analysis pipeline render to its expected output in a clean environment?" The tiered model proposed in Section 5 is appropriate for compendia but adds unnecessary complexity for tool packages.

The framework's default workflow produced different real-world behaviors in the two cases despite identical configuration. Tool package CI was failing for reasons related to renv-in-container mechanics; compendium CI was passing despite the same configuration because the test surface happened not to expose those failure modes.

9.5 Blog post sub-repos and the limits of file-presence detection

The `qblog/posts/` tree, a directory of fifty-plus zzcollab-scaffolded sub-repos that accompany Quarto blog posts, surfaced a workspace category whose CI requirements differ from both the tool-package and LaTeX-compendium types discussed in 9.4. Inspection of one representative (`penguins1zzcollab`) found that the project has a complete R-package skeleton (DESCRIPTION, NAMESPACE, R/, tests/) but its primary deliverable is a Quarto document rendered to HTML, not a distributable package and not a LaTeX-rendered paper. The repository's existing `blog-render.yml` workflow (which had been passing) renders the document inside a Dockerfile-built container, on changes to `analysis/report/`, `renv.lock`, or the Dockerfile.

The first revision of the tiered model applied to this repository included Tier 2 (`R CMD check` via `check-r-package@v2`). That job failed at the `R CMD build` stage with

```
cp: cannot stat 'penguins1zzcollab/analysis/report/figures':
No such file or directory
ERROR
copying to build directory failed
```

The cause was an empty `analysis/report/figures` directory referenced in some build manifest. The deeper cause was the misapplication of `R CMD check` to a project that is not actually being shipped as a package: the package machinery tries to assemble a build tarball that makes sense only for a CRAN-bound artifact, and chokes on the compendium-style directories that have no place in a tarball.

The file-presence signals that initially placed this project in the "compendium" category (DESCRIPTION + R/ + tests/) underdetermined the workspace type. zzcollab scaffolds a uniform R-package skeleton across all paradigms, including blog posts, so the presence of these files is not a reliable indicator that `R CMD check` is appropriate. A more discriminating signal is found in the YAML header of the primary document, where `document-type: "blog"` (or equivalent) declares intent.

The corrected pattern for a Quarto blog post is **Tier 1 only** for the always-on validation, plus the existing `blog-render.yml` for the path-filtered render. There is no Tier 2 because nothing is being shipped as a package. Section 11 records this as a fourth workspace category with its own CI footprint.

9.6 Report file naming variety in zzcollab compendia

A survey of eighteen compendium projects in `~/prj/alz/` (a research project tree using zzcollab) found multiple naming conventions for the primary manuscript file in `analysis/report/`:

Filename convention	Count	Examples
`report.Rmd`	6	mci, mcid-cdr, psp, ptsd-diabetes-mediation
`manuscript.Rmd`	2	medications-progression, age
Custom `*_whitepaper.Rmd`	1	world-backwards
Non-standard location (`docs/`)	1	murray-yeilim
No report file	5	scaffolding only, awaiting content

All eighteen are RMarkdown projects targeting LaTeX/PDF output. Zero Quarto (`.qmd`) projects in this tree, in contrast with the qblog sub-repo population, where `.qmd` is the standard.

Two relevant observations:

1. **The canonical zzcollab `render-report.yml` template already handles RMarkdown naming variety.** It tries `analysis/report/report.Rmd`, then `manuscript.Rmd`, then `main.Rmd`, then falls back to the largest `.Rmd` in the directory. Naming variety within RMarkdown is not the gap.

2. **The canonical template does not handle `.qmd` files.** The detection step in the canonical recognises `.Rmd` only. Compendia that use Quarto would not be matched and the fallback path returns no manuscript. This is the actual file-detection gap.
3. **Stale variants of `render-report.yml` are in circulation that *do* hardcode `report.Rmd`** (the `c1e956e...` MD5 variant observed in pznblastanalysis and ptsd-diabetes-mediation, which originated in the `templates/.github/` tree deleted earlier in this work). Projects on this stale variant fail silently for any manuscript not literally named `report.Rmd`.

The corrective action is to extend the canonical template's detection to include `.qmd` files and to ensure the stale variants are upgraded via `zzc doctor` to the current canonical. The first is patched in this work; the second is a separate cleanup pass.

9.7 Tier 1 "pass" is not "consistent": verification on penguins1zzcollab

After the Tier-1-only workflow was deployed to penguins1zzcollab and the run completed, the GitHub Actions UI showed the job in green ("success", 1m14s). Inspection of the job log revealed that `renv::status()` had reported a substantial inconsistency in the project state:

```
The following package(s) are in an inconsistent state:
 package    installed  recorded  used
 askpass    y          n         y
 backports  y          n         y
 base64enc  y          n         y
 ... (continues for many more packages)
```

Translation: dozens of packages were installed in the renv library and used by the source code, but were not recorded in `renv.lock`. The lockfile pinned twelve packages while the project actually depended on roughly ten times that count. This is exactly the silent reproducibility failure mode the validate tier is supposed to catch.

The reason the workflow nonetheless reported "pass" is that `renv::status()` does not raise an error when it finds inconsistency. It prints findings and returns a list, and the calling shell wrapper exits 0 regardless. A reader scanning only the green checkmark in GitHub Actions would conclude the project is healthy.

The fix is to capture the return value of `renv::status()` and exit explicitly if synchronization fails:

```r
status <- renv::status()
if (!isTRUE(status$synchronized)) {
  cat("\nrenv reports the project is not synchronized.\n",
      "Run renv::snapshot() locally and commit the updated ",
      "renv.lock.\n", sep = "")
  quit(status = 1)
}
```

This pattern was added to penguins1zzcollab after the verification run and should be the canonical Tier 1 implementation in the framework's templates. Section 5's Tier 1 sketch and Section 11's patterns should be read as including this synchronization check.

The broader lesson is one of CI hygiene: a tier whose intent is to catch a class of regression must be configured to fail when that regression is detected, not merely to print evidence of it. The visible signal in CI dashboards is the job exit status, not the log content. A green dashboard with red findings in logs is operationally indistinguishable from an absent CI check.

9.8 Multi-repo verification: zero passing, all informative

After the single-repo verification on `penguins1zzcollab` (Section 9.7), the new-generation pattern was deployed across five additional repositories spanning all three observed workspace categories:

Repo	Workspace type	Workflow change
`res/02-adaptive-alloc-survival`	LaTeX compendium	replace v2.4.0-A with `r-package.yml` (validate+check) + `render-report.yml` (rmarkdown + tinytex)
`alz/15-mcid-cdr`	LaTeX compendium	same
`alz/04-mci`	LaTeX compendium	retroactive `r-version: 'renv'` fix to existing tiered model
`qblog/14-penguins1zzcollab`	Quarto blog post	`r-version: 'renv'` fix
`qblog/13-palmerpenguinsregression`	Quarto blog post	replace v2.4.0-A with Tier-1-only `r-package.yml`; retain existing `blog-render.yml`; delete misapplied `render-report.yml`

Of these five, **zero passed CI on first run**. Every failure was project-side and informative; none were CI-infrastructure noise. The failures distributed across three categories, each of which adds material to the analysis recorded earlier in this section.

Failure category 1: R-version mismatch (newly observed). The default `r-version: 'release'` in `setup-r@v2` resolved to R 4.6.0 at the time of the verification (May 2026). The lockfiles in the target repositories all pinned R 4.4.2. Packages compiled for R 4.4 failed to load against R 4.6's headers, producing errors such as `R_NamespaceRegistry was not declared in this scope` and `R_ext/PrtUtil.h: No such file or directory`. The fix is to set `r-version: 'renv'` on `setup-r@v2`, which reads the R version from `renv.lock` and matches the lockfile's pinned version. This is a one-line addition per setup-r block:

```yaml
- uses: r-lib/actions/setup-r@v2
  with:
    use-public-rspm: true
    r-version: 'renv'
```

The new-generation template should include this line. Without it, any zzcollab project whose lockfile pins an R version older than the current release will fail in the same way once R 4.7 is released, and so on.

Failure category 2: Posit binary aging-out is a recurring pattern. Two of the five repositories (`res/02` and `alz/04-mci`) failed at `S7 0.2.1-1` download. The `-1` suffix is a Posit binary build identifier. This is structurally identical to the `Rcpp 1.1.1-1` failure documented in Section 9.3: the lockfile records a specific binary build; that exact build ages out of Posit Package Manager's snapshot retention; subsequent restores cannot fetch it. The Rcpp instance was treated in 9.3 as an isolated case; the S7 recurrence indicates this is a class of failure rather than a one-off. Section 9.1's recommendation to pin `R$Repositories.URL` to a date-stamped Posit URL with the correct OS segment (`__linux__/noble/<date>`) is reinforced as the structural fix; the immediate workaround is `renv::snapshot()` locally and a fresh commit so the lockfile records currently- available builds.

Failure category 3: R CMD check WARNINGs surface latent package-quality regressions. `alz/15-mcid-cdr` failed at `R CMD check` with

```
Status: 1 WARNING, 4 NOTEs
Undocumented code objects:
  'calculate_weighted_mean_ci' 'ci95'
Consider adding
  importFrom("stats", "qt")
```

`r-lib/actions/check-r-package@v2` errors on WARNINGs by default. The previous v2.4.0-A workflow used a hand-rolled `rcmdcheck` call with `error_on = 'error'`, which let WARNINGs pass. The documentation gaps were therefore present in the package long before the migration; the new pattern surfaces them rather than masking them. This is empirical evidence for Section 9.7's hygiene principle: the visible signal in CI dashboards is the job exit status, not the log content, and a workflow whose error-on threshold is set permissively cannot serve as a quality gate.

Failure category 4: Tier 1 synchronized check verified at scale. Three of the five failures were the explicit `!isTRUE(status$synchronized)` check from Section 9.7 firing (`penguins1zzcollab`, `palmerpenguinsregression`, `mcid-cdr`'s validate tier). This validates the Section 9.7 design at scale: the strict synchronized check produces honest red signals rather than misleading green checkmarks for projects in lockfile-drift state. None of the three projects had been detecting their drift under the prior CI; all three are detecting it now.

Empirical assessment. Zero of five passing is the correct result for the current project state. The same five repositories under the previous v2.4.0-A workflow were either passing (`mci`, `mcid-cdr`, `adaptive-alloc-survival`) or failing for opaque infrastructure reasons (`palmerpenguinsregression`, `penguins1zzcollab`). The new-generation pattern flips this distribution: every failure now points to a real project-side regression that the previous CI was masking, and the failure messages are actionable. The framework's CI now measures what its purpose is supposed to measure.

9.9 Empirical reliability favors simpler workflows

Across the projects audited during this work:

Workflow	Approach	Pass rate
v2.2.0	`rocker/verse:latest`, DESCRIPTION-based, no renv in CI	3 of 4 (75%)
v2.4.0-A	Pinned `rocker/tidyverse`, renv-restore-in-container	1 of 5 (20%)

The newer workflow's narrower scope and increased complexity did not buy reliability. The simplest reliable thing observed was the `setup-renv@v2` action pattern used in zzobj2fig's separate `pkgdown.yaml` and `test-coverage.yaml` workflows, which were never failing during the period the main workflow was. That pattern is roughly twenty lines of YAML, requires no Docker container, and handles caching automatically.

10. Updated Implementation Pattern

The recommendations in Section 8 should be revised in light of Sections 9.4 through 9.6. The framework should distinguish project type at scaffolding time and emit a workflow template appropriate to the type. At least four workspace types have been observed in practice (tool package, LaTeX compendium, Quarto compendium, Quarto blog post), each with a distinct CI footprint catalogued in Section 11. The two examples in this section (tool package and compendium) cover the dominant cases; Section 11.1 records the full detection logic.

10.1 Tool packages

For projects whose deliverable is the package itself, the workflow should follow the canonical R-package CI pattern:

```yaml
name: R-CMD-check
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: r-lib/actions/setup-r@v2
        with:
          use-public-rspm: true
      - uses: r-lib/actions/setup-renv@v2
      - uses: r-lib/actions/check-r-package@v2
```

`setup-renv@v2` handles renv installation, lockfile restore, and GitHub Actions caching. `use-public-rspm: true` uses the Posit Public Package Manager for binary packages on Linux. No Docker container is needed in CI; the container remains useful for local development.

Optionally, this can be extended to a matrix across R versions and operating systems at no significant additional cost.

10.2 Compendia

For projects whose deliverable is a rendered report, the tiered model from Section 5 is appropriate, but each tier should be built on the same `setup-renv@v2` pattern rather than a Docker container. The render tier (Tier 3) adds `setup-pandoc@v2` and `setup-tinytex@v2` for documents that require LaTeX, and uploads the rendered output as a build artifact:

```yaml
render:
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: r-lib/actions/setup-pandoc@v2
    - uses: r-lib/actions/setup-tinytex@v2
    - uses: r-lib/actions/setup-r@v2
      with:
        use-public-rspm: true
    - uses: r-lib/actions/setup-renv@v2
    - shell: Rscript {0}
      run: rmarkdown::render('analysis/report/report.Rmd')
    - uses: actions/upload-artifact@v4
      with:
        name: report
        path: analysis/report/report.pdf
```

The validate and check tiers compose the same way, replacing the render step with `renv::status()` and `check-r-package@v2` respectively.

10.3 Detecting project type at scaffolding

`zzc analysis` and `zzc package` are reasonable distinct entry points; the framework already exposes paradigm flags for analysis, modeling, and other variants. The workflow template emitted should follow the paradigm chosen at init, with `analysis` and `manuscript` paradigms receiving the tiered compendium template and other paradigms receiving the tool-package template. A project that switches type later can be re-templated by the existing `zzc doctor` upgrade path, which the patches in this work restructured to handle full-content workflow replacement rather than version-stamp bumping.

11. Workspace Types and Their CI Patterns

The implementations recorded in this work (zzobj2fig, mci, penguins1zzcollab) span enough variation to support a small typology of zzcollab workspace types. Each type has a distinct CI footprint. The table below maps each observed workspace category to its appropriate CI pattern.

Workspace type	Deliverable	Tier 1 (validate)	Tier 2 (check)	Tier 3 (render)	Example
Tool package	Package	yes	yes	(none)	zzobj2fig
Compendium (LaTeX)	PDF via xelatex	yes	yes	rmarkdown + tinytex	mci
Compendium (Quarto)	HTML or PDF document	yes	yes	quarto render	(anticipated)
Quarto blog post	**Rendered HTML**	**yes**	**NO**	**quarto render (path-filtered)**	**penguins1zzcollab**
Minimal package	Package	yes	yes	(none)	(anticipated)

Four observations follow.

Tier 1 is workspace-invariant; Tiers 2 and 3 are not. The validate job is identical across all five rows: install renv, run `renv::status()`. Tier 2 (`R CMD check`) is appropriate when the project is being built or shipped as a package, but is actively harmful for blog posts where the package skeleton is scaffolding rather than the deliverable. Tier 3 differs by render target.

Tool packages should omit the render tier. Adding a render job to a project with no document to render produces a no-op job whose only effect is workflow clutter. This was the mistake in the first revision applied to zzobj2fig (Section 9.4): the tiered template was applied to a tool package, where Tier 3 had no work to do. The corrected approach for tool packages (Section 10.1) drops Tier 3 entirely.

Quarto blog posts should omit the check tier. This was the mistake in the first revision applied to `penguins1zzcollab` (Section 9.5): the tiered template was applied to a project that is not a package, and Tier 2 failed at `R CMD build` for reasons that have nothing to do with whether the blog post is correct. The corrected approach is Tier 1 (lockfile validation) plus a path-filtered Tier 3 (render) and no Tier 2 at all.

Hybrid workspaces are common in zzcollab output and require discriminating signals. Both `penguins1zzcollab` (a blog post) and a hypothetical compendium with `R/` populated would have identical file presence in DESCRIPTION, NAMESPACE, R/, and tests/. File presence alone underdetermines the workspace type. The discriminating signal is the YAML header of the primary document, discussed in Section 11.1.

11.1 Detection at scaffolding time

The framework can choose the appropriate template by inspecting files and file headers at `zzc analysis` or `zzc doctor` time. The ordering matters: blog-post detection should fire before compendium detection because the file-presence overlap is the source of misclassification (Section 9.5). File-extension detection should accept any `.Rmd` or `.qmd` in `analysis/report/` rather than literal filenames, because zzcollab projects in the wild use `report.Rmd`, `manuscript.Rmd`, `main.Rmd`, and various custom names interchangeably (Section 9.6).

1. **Blog post pattern.** `analysis/report/index.qmd` (or top-level `index.qmd`) exists AND the YAML header contains `document-type: "blog"` or the project is part of a Quarto blog tree. Emits `blog-render.yml` (path-filtered render) plus a Tier 1 validation workflow. No `R CMD check`.
2. **LaTeX compendium pattern.** Any `.Rmd` exists in `analysis/report/` and its YAML header contains `output: pdf_document` (or equivalent LaTeX-target output). Tiered model with `rmarkdown::render` and `setup-tinytex@v2`. Examples in alz tree: mci, mcid-cdr, psp, medications-progression (the last using `manuscript.Rmd` rather than `report.Rmd`).
3. **Quarto compendium pattern.** Any `.qmd` exists in `analysis/report/` without a `document-type: "blog"` header, targeting HTML or PDF via Quarto. Tiered model with `quarto render`.
4. **Tool package pattern.** None of the above; only `R/`, `tests/`, `man/` populated. `setup-renv@v2` plus `check-r-package@v2`. No render tier.

A `zzc doctor` upgrade can re-detect on existing projects and swap the template, using the full-content replacement path introduced earlier in this work rather than a stamp-only bump.

The canonical `render-report.yml` template (after the patch applied in this work) already enumerates `report.Rmd`, `manuscript.Rmd`, `main.Rmd`, plus `.qmd` analogues, plus a fallback to the largest matching file in `analysis/report/`. So the file-name signal is robust at the workflow level. The discrimination question is one tier higher: which template to emit, not which file the template should match.

11.2 The two-workflow split is an existing zzcollab pattern

The two-workflow split (one cheap always-on workflow plus one expensive path-filtered workflow) is not a new design proposed by this work. It is an existing pattern already in zzcollab's template set, applied inconsistently across deployed projects.

Two existing zzcollab template files implement the split:

- `templates/workflows/r-package.yml` – the always-on package check; runs on every push and pull request.
- `templates/workflows/render-report.yml` – the path-filtered manuscript render; triggers only on changes under `analysis/**`, `R/**`, `DESCRIPTION`, or itself.

In the alz tree of eighteen compendium projects, three (pznblastanalysis, ptsd-diabetes-mediation, pmsimstats-ng) deploy both workflows together, three different MD5 variants of `render-report.yml` in circulation. The remaining fifteen have only `r-package.yml`, even when their `analysis/report/` directory contains a manuscript that would benefit from automated render verification.

Two corollaries follow:

1. **The pattern is sound; the deployment is partial.** The two blog-post workflows in `qblog/posts/` (the validation workflow added in this work plus the existing `blog-render.yml`) instantiate the same pattern with a Quarto-rendering Tier 3. The framework's recommended posture is to deploy both workflows for any workspace that has a renderable document, regardless of whether the renderer is `rmarkdown::render` or `quarto render`.

2. **Stale variants need re-templating.** The three variants of `render-report.yml` in circulation reflect the same template-drift problem documented in Section 9 for `r-package.yml`. The same `zzc doctor` full-content replacement path applies. Compendia with stale variants should be re-templated to the canonical `render-report.yml` (which after the patches in this work handles both `.Rmd` and `.qmd` files and accepts naming variety).

The contribution of this work in 11.2 is therefore not the introduction of a new pattern but the explicit recognition that the existing pattern should be the framework default for any compendium, including blog posts, and that it should be propagated uniformly rather than left to per-project judgment at scaffolding time.

11.3 What stays in the framework

Across all workspace types, the framework's contribution is unchanged: the Five Pillars (Dockerfile, renv.lock, .Rprofile, source code, data) jointly support local development reproducibility. The Docker container remains useful as a development environment even though it is not used in CI. The renv lockfile, with the URL fix from Section 9.1, remains the package-pin mechanism. The .Rprofile (with `ZZCOLLAB_AUTO_RESTORE=false` honored in CI) remains the session bootstrap.

What changes is the CI template emitted by `zzc`, which becomes type-aware rather than uniform. The same five pillars support five different downstream CI configurations, all reusing the `r-lib/actions/setup-renv@v2` foundation.

12. Conclusion

The question this paper addresses is not "how do we make CI pass," but "what should CI check, given what zzcollab is for." The framework's purpose is research-compendium reproducibility, but its output spans at least four workspace categories: tool package, LaTeX manuscript compendium, Quarto compendium, and Quarto blog post. Each has a distinct CI footprint, and the framework's single CI default conflates them. Recognising this distinction is itself part of the contribution.

The conventional R-package CI idiom inherited from the broader R ecosystem is a reasonable starting point but did not align with the framework's stated purpose for compendium projects, and is actively wrong for blog posts where the package skeleton is scaffolding rather than the deliverable. A tier model (environment integrity, code correctness, end-to-end reproduction) addresses the regressions a compendium author actually cares about. For tool packages, the conventional `setup-renv@v2` pattern is the right default; for compendia, the same pattern with a render tier added; for blog posts, the same pattern with the check tier removed.

The recommendation for current zzcollab projects is incremental: fix the lockfile URL pinning first, since that improves reliability without restructuring and addresses a silent reproducibility failure mode that affects all projects regardless of CI strategy; then distinguish project type and apply the appropriate template; and only then consider the secondary question of how aggressively to verify outputs against recorded baselines.

Several findings from implementation were not anticipated in the original analysis. The lockfile URL is itself the most consequential defect, silently producing version drift on every restore. The curl-CLI warning is a misleading symptom rather than a root cause. The .Rprofile auto-restore can race the workflow's explicit restore unless `ZZCOLLAB_AUTO_RESTORE=false` is set in CI environment.

Report file naming is not standardised in zzcollab compendia (`report.Rmd`, `manuscript.Rmd`, custom names, plus `.qmd` analogues all observed in the alz and qblog trees), and the canonical `render-report.yml` template needed extension to handle `.qmd` in addition to its existing multi-name `.Rmd` detection. The two-workflow split (always-on validation plus path-filtered render) is not a new design but an existing zzcollab pattern, partially deployed across the alz tree and inconsistent in version. The framework's contribution is not a new pattern but the explicit recognition that the existing pattern should be propagated uniformly and parameterised on workspace type.

These are now part of the record.

Rendered on 2026-05-06 at 17:52 PDT. Source: ~/prj/sfw/07-zzcollab/zzcollab/docs/ci-strategy-tiered-model.md